Script-Controlled ACL – Restrict Record Access Based on Field Value

Phase 7- Project Executable Files

Date: 30 September 2026

Team ID: 6ab3ed08a238dc999eb3fda5

Project Name: Script-Controlled ACL – Restrict Record Access Based on Field Value

Maximum Marks: 3 Marks

Project Executable Files

This section provides the executable/deployable project information and the instructions required for the evaluator to configure and run the ServiceNow ACL project.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	ServiceNow project configuration / ACL implementation	Yes
2	README / Setup Guide (instructions to configure and test the project)	Yes
3	Requirements / dependency information	NA
4	Database schema / table and field configuration	Yes
5	Environment configuration	NA
6	Deployed application URL	To be provided
7	APK / Executable binary	NA
8	Dockerfile / Containerization config	NA
9	Test / verification steps	Yes
10	Demo video or walkthrough	To be provided if required

Step 2: File / Folder Structure

The project is implemented as a ServiceNow configuration-based project. The main project components are organized as follows:

- ServiceNow Instance
- Users: EEE User and Administrator
- Custom Roles: bb1, bb2, bb3, bb4
- Custom Table: u_institution_details
- Fields: Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, Description
- ACLs: Read, Create, Write, and Delete

- Script-controlled Read ACL
- Test / verification scenarios

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	ServiceNow instance URL: _____
Login Credentials (Demo)	Admin credentials / demo credentials to be provided securely
Platform / Hosting Provider	ServiceNow
Repository Link	Not applicable / To be provided
Demo Video Link	To be provided if required

Step 4: Run Instructions

1. Log in to the ServiceNow instance with admin access.

2. Navigate to User Administration → Users and create the EEE User.

3. Create the custom roles bb1, bb2, bb3 and bb4 and assign the required roles to the test user.

4. Navigate to System Definition → Tables and create the custom table:

Table Label: Institution Details

Table Name: u_institution_details

5. Add the required fields: Student Roll Number (Auto Number), Student Name (Reference – User), Faculty Name (Reference – User), Branch (Choice: ECE, EEE, CSE), Email (String), Phone Number (String), and Description (Multi String).

6. Create multiple records with different Branch values such as ECE, EEE and CSE.

7. Navigate to System Security → Access Control (ACL) and elevate the Security_admin role.

8. Create the READ ACL for u_institution_details. Set Type = record, Operation = read, Active = true, add role bb1, set the data condition as Branch is EEE, and enable Advanced.

9. Use the following script in the Advanced Script field:

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }

  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')){
    return true;
  }
```

```
  // Deny access for all others
  return false;
})();
```

10. Create the CREATE ACL for u_institution_details with Operation = create and role bb2.

11. Create the WRITE ACL for u_institution_details with Operation = write and role bb3.

12. Create the DELETE ACL for u_institution_details with Operation = delete and role bb4.

13. Impersonate the EEE User and open the u_institution_details list.

14. Verify that a user with bb1 can view EEE branch records, while users without the required role cannot view records.

15. Verify that the administrator can view all records regardless of branch.

16. Verify the cumulative Create, Write and Delete permissions using users with the required roles.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	ServiceNow instance must be available and accessible.	Use an active ServiceNow instance.
2	Security_admin role must be elevated before ACL configuration.	Elevate the role before creating or modifying ACLs.
3	The custom table and field names must match the configured ACLs.	Verify u_institution_details and the Branch field.
4	Demo URL and credentials are instance-specific.	Provide the evaluator with valid demo access separately.

Project Outcome

By completing this project, learners gain a comprehensive understanding of how script-controlled READ, WRITE, CREATE, and DELETE ACLs enforce record-level security, how user roles and record field values are evaluated together to make access decisions, and how ACLs protect sensitive data from unauthorized viewing, modification, creation, and deletion across forms, lists, and Service Portal interfaces.